

Állatorvosi klinika kezelőrendszer



Debreceni SZC Mechwart András Gépipari és Informatikai Technikum

Készítette:

Balogh Bence
Csomós János
Ráczempek Lajos

Szoftverfejlesztő és -tesztelő képzés

Debrecen, 2025-2026-os tanév

Tartalomjegyzék

1. Bevezetés	3
1.1 A téma ismertetése	3
1.2 Témaválasztás indoklása	3
1.3 A rendszer célja és célcsoportja	4
1.4 Fő funkciók	4
2. Követelményelemzés	5
2.1 A rendszerrel szemben támasztott követelmények	5
3. Fejlesztői környezet	6
3.1 Hardver- és szoftverigény	6
4. Alkalmazott technológiák	7
5. Rendszerarchitektúra	8
6. Backend fejlesztés	9
6.3 Backend kódrészlet - Owner és Pet modellek	10
7. Adatmodellek	11
7.4.1 Backend kódrészlet - Appointment modell	12
7.9 Adatbázis-kapcsolati diagram	15
8. REST API	17
9. Felhasználói hitelesítés és biztonság	19
9.3 Backend kódrészlet - foglalási jogosultságok	20
9.4 Backend kódrészlet - REST API és JWT végpontok	21
10. Adminisztrációs felület	22
11. Frontend fejlesztés és felhasználói felület	23
12. A program használata	30
12.4 Felhasználói folyamat és jogosultságok	31
13. Telepítési és futtatási útmutató	32
14. Tesztdokumentáció	33
15. Továbbifejlesztési lehetőségek	35
16. Összefoglalás	36

1. Bevezetés

1.1 A téma ismertetése

A vizsgaremek egy webalapú állatorvosi klinikai kezelőrendszer, amely egy rendelő mindennapi adminisztrációját támogatja. A rendszer célja, hogy egyetlen felületen legyenek kezelhetők a tulajdonosok és kisállataik adatai, az időpontfoglalások, az állatorvosok, az oltások, a kezelési előzmények és a gyógyszeres receptek.

A backend Django és Django REST Framework alapokra épül, az adatokat SQLite adatbázis tárolja, a frontend pedig REST API-n keresztül éri el a szükséges erőforrásokat. A rendszer elkülöníti az adatkezelési és megjelenítési feladatokat, így a két réteg egymástól függetlenül is fejleszthető.

1.2 Témaválasztás indoklása

Az állatorvosi rendelőkből egyszerre sok, egymással összefüggő információt kell nyilvántartani. Egy kisállathoz tulajdonos, időpontok, vizsgálatok, oltások és gyógyszeres kezelések kapcsolódhatnak. Ezek papíron vagy több különálló rendszerben történő kezelése nehezen áttekinthető. A projekt ezért egy olyan egységes, könnyen használható megoldást mutat be, amely a rendelői folyamatokat strukturált adatmodellre építve kezeli.

A vizsgaremek szakmai szempontból is megfelelő összetettségű: adatbázis-kezelést, REST API-t, autentikációt, jogosultságkezelést, képfeltöltést, frontend-backend kommunikációt és adminisztrációs felületet egyaránt tartalmaz.

1.3 A rendszer célja és célcsoportja

A rendszer elsődleges célja a klinikai adminisztráció egyszerűsítése, ugyanakkor nem kizárólag adminisztrátori használatra készült. A rendelő munkatársai mellett civil állattulajdonosok is regisztrálhatnak, saját kisállataikat rögzíthetik, valamint időpontot kezdeményezhetnek a rendelőbe.

- állattulajdonosok, akik saját fiókot és kisállat-profilt kezelnek;
- állatorvosi rendelők és kisebb klinikák adminisztratív munkatársai;
- állatorvosok, akiknek szükségük van a kezelések és oltások áttekintésére;
- rendszergazdai jogosultsággal rendelkező felhasználók;
- a fejlesztési projektet tesztelő vagy értékelő személyek.

1.4 Fő funkciók

- Felhasználói regisztráció és JWT-alapú bejelentkezés.
- Saját kisállatok adatainak rögzítése és kezelése.
- Időpontfoglalás kezdeményezése; az új foglalás alapértelmezetten függőben van.
- A foglalások adminisztrátori elbírálása és státuszkezelése.
- Tulajdonosok, állatorvosok, kezelések, oltások és gyógyszeres receptek nyilvántartása.
- Képfeltöltés, Django adminisztráció és reszponzív React felhasználói felület.
- Tokenfrissítés és kijelentkezés, jogosultság alapján szűrt adathozzáférés.

2. Követelményelemzés

2.1 A rendszerrel szemben támasztott követelmények

Összetevő	Minimum követelmény	Ajánlott
Operációs rendszer	Windows 10 64 bit	Windows 11 64 bit
Processzor	2 magos, 2,0 GHz	4 magos vagy jobb
Memória	4 GB RAM	8 GB RAM vagy több
Tárhely	2 GB szabad hely	5 GB vagy több
Kijelző	1280×720	1920×1080
Böngésző	Aktuális Chrome / Edge / Firefox	Legfrissebb stabil verzió
Fejlesztői futtatás	Python 3, Node.js, npm, Git	Aktuális stabil/LTS kiadások

A rendszer kialakításánál elsődleges szempont volt az áttekinthető kezelőfelület, az adatok strukturált tárolása, a biztonságos hozzáférés és a későbbi bővíthetőség. A kliens és a szerver külön réteggént működik, így a felhasználói felület és a háttérrendszer egymástól elkülönítve fejleszthető és tesztelhető.

A rendszer használatához modern webböngésző, stabil helyi vagy hálózati kapcsolat, valamint a szerveroldali és kliensoldali környezet megfelelő telepítése szükséges. A védett adatok kizárólag bejelentkezés után érhetők el.

- Átlátható, egységes és magyar nyelvű kezelőfelület.
- Hitelesített hozzáférés a védett klinikai adatokhoz.
- A backend és frontend külön indítható, moduláris felépítése.
- A rendszer Windows és macOS fejlesztői környezetben is futtatható.
- Könnyen telepíthető fejlesztői környezet.

3. Fejlesztői környezet

A projekt kliens-szerver felépítésben készült. A backend Python és Django környezetben, a frontend React és Vite segítségével fut. A forráskód verziókezelésére Git használható, a fejlesztéshez pedig Visual Studio Code vagy más modern kódszerkesztő megfelelő. A rendszer elsődleges bemutatási környezete Windows, ugyanakkor macOS alatt is futtatható.

Terület	Eszköz / technológia	Szerep
Backend	Python	Szerveroldali programozási nyelv
Backend	Django	Webes keretrendszer és ORM
API	Django REST Framework	REST végpontok és serializáció
Adatbázis	SQLite	Fejlesztési adatbázis
Biztonság	Simple JWT	Access és refresh token kezelés
Média	Pillow	Képfeltöltés és képfeldolgozás
Admin	Django Admin	Beépített adminisztráció
Frontend	React 19 / React Router	Felhasználói felület és kliensoldali útvonalkezelés
Frontend futtatás	Vite / Node.js / npm	Fejlesztői szerver és csomagkezelés

3.1 Hardver- és szoftverigény

A felhasználói oldal önmagában nem igényel nagy számítási teljesítményt, ezért átlagos irodai vagy otthoni számítógépen is használható. Fejlesztői környezetben a backend és a frontend egyszerre fut, ezért legalább 4 GB RAM és néhány gigabájt szabad tárhely szükséges. A fenti táblázat a Windows alapú minimum konfigurációt foglalja össze.

4. Alkalmazott technológiák

4.1 Django és Django REST Framework

A Django biztosítja az adatmodelleket, az ORM-et, a szerveroldali konfigurációt és a beépített adminisztrációs felületet. A Django REST Framework segítségével az adatmodellek REST végpontokon keresztül érhetők el. A végpontok kialakításánál ModelViewSet és DefaultRouter megoldások egyszerűsítik a CRUD műveletek kezelését.

4.2 SQLite

A projekt SQLite adatbázist használ. Fejlesztési és vizsgaprojekthez előnye, hogy külön adatbázis-szerver telepítése nélkül, egy fájlban tárolhatók az adatok. A Django ORM-en keresztül a későbbi adatbázisváltás is kezelhető.

4.3 JWT és Simple JWT

A hitelesítés JWT tokenekkel történik. Sikeres bejelentkezéskor access és refresh token jön létre. Az access token használható a védett végpontok eléréséhez, a refresh token pedig új access token kérésére.

4.4 Pillow és képfeltöltés

A kisállatokhoz fénykép is rendelhető. A backend ImageField mezőt használ, a képek kezelését a Pillow könyvtár támogatja.

4.5 React, Vite és kliensoldali könyvtárak

A felhasználói felület React 19 alapú. A projekt fejlesztői szerverét és build folyamatát Vite biztosítja, az útvonalkezeléshez react-router-dom, az ikonokhoz lucide-react csomag használatos. A komponensek funkció szerint oldalakra, elemekre és szolgáltatásokra tagolódnak.

5. Rendszerarchitektúra

Az alkalmazás három fő rétegre bontható: felhasználói felület, REST API és adatbázis. A frontend HTTP kéréseket küld a backendnek. A backend elvégzi a hitelesítést, az üzleti logikát és az adatbázis-műveleteket, majd JSON formátumú választ ad vissza.

Frontend → REST API → Django modellek / ORM → SQLite adatbázis

A rétegek szétválasztása azért előnyös, mert a felhasználói felület változtatása nem igényli az adatbázis teljes újratervezését, és az API más kliensből is használható lehet.

5.1 Adatáramlás példája - időpontfoglalás

1. A bejelentkezett állattulajdonos kiválasztja a saját kisállatát, az állatorvost, a dátumot és a foglalás okát.
2. A frontend elküldi a foglalás adatait a backend megfelelő API-végpontjára.
3. A backend ellenőrzi a hitelesítést és azt, hogy a kisállat valóban a felhasználóhoz tartozik.
4. Az Appointment rekord normál felhasználónál automatikusan pending (függőben) státusszal jön létre.
5. Az adminisztrátor a foglalás státuszát elfogadottra, befejezettre vagy lemondottra módosíthatja.

6. Backend fejlesztés

A backend feladata az állatorvosi rendelő működéséhez szükséges adatok kezelése, tárolása és a frontend számára REST API-n keresztül történő biztosítása. A rendszerben külön modellek készültek a fő üzleti objektumokhoz.

6.1 Backend modulok

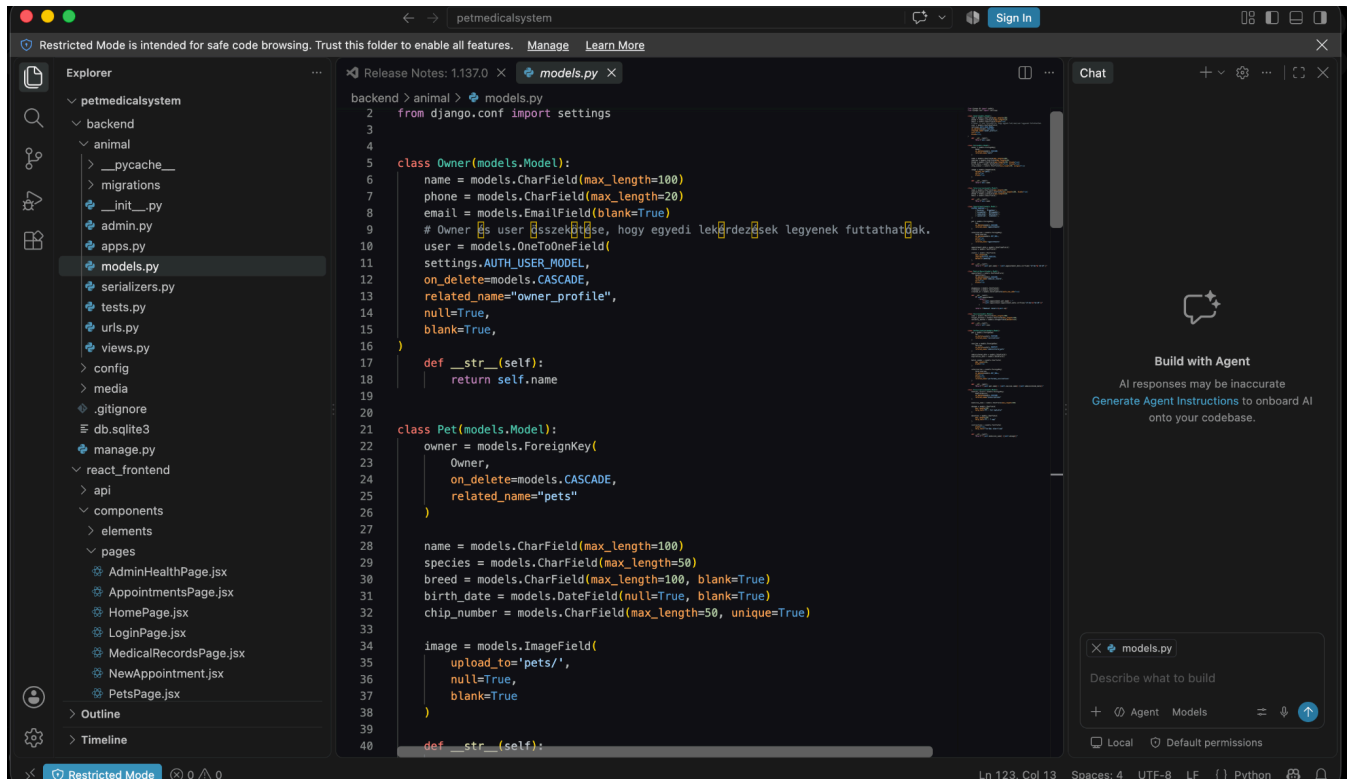
- Owner - tulajdonosok
- Pet - kisállatok
- Veterinarian - állatorvosok
- Appointment - időpontok
- MedicalRecord - orvosi dokumentáció
- Vaccine - vakcinatípusok
- PetVaccination - oltási események
- Prescription - gyógyszeres receptek

6.2 Kapcsolatok

A modellek között Foreign Key, One-to-One és szükség esetén Many-to-Many kapcsolatok alkalmazhatók. A rendszerben például egy Owner rekordhoz több Pet tartozhat, az időpont pedig kisállathoz és állatorvoshoz kapcsolódik.

6.3 Backend kódrészlet - Owner és Pet modellek

Az Owner modell OneToOne kapcsolatban áll a Django felhasználói fiókjával, a Pet modell pedig ForeignKey kapcsolaton keresztül tartozik egy Owner rekordhoz. Ez teszi lehetővé, hogy egy regisztrált állattulajdonos több saját kisállatot kezeljen.



10. ábra - Owner és Pet modellek kapcsolata a models.py fájlban

7. Adatmodellek

7.1 Owner - tulajdonos

Az Owner modell az állatok tulajdonosainak alapvető adatait tárolja. A modell fő mezői: név, telefonszám és e-mail-cím. Egy tulajdonoshoz több kisállat tartozhat, ezért az Owner és Pet között egy-a-többhöz kapcsolat van.

Mező	Jelentés
name	Tulajdonos neve
phone	Telefonszám
email	E-mail-cím

7.2 Pet - kisállat

A Pet modell a klinikai rendszer egyik központi eleme. A kisállathoz kapcsolódnak a tulajdonosi adatok, időpontok, kezelési előzmények, oltások és receptek.

Mező	Jelentés
name	Kisállat neve
species	Faj
breed	Fajta
birth_date	Születési dátum
chip_number	Chip szám
owner	Tulajdonos kapcsolata
image	Fénykép

7.3 Veterinarian - állatorvos

A Veterinarian modell az állatorvosok adatait tárolja. A modell név, szakterület, telefonszám és e-mail-cím adatokat tárol. Az állatorvosok időpontokhoz és oltási eseményekhez rendelhetők.

Mező	Jelentés
name	Állatorvos neve
specialization	Szakterület
phone	Telefonszám
email	E-mail-cím

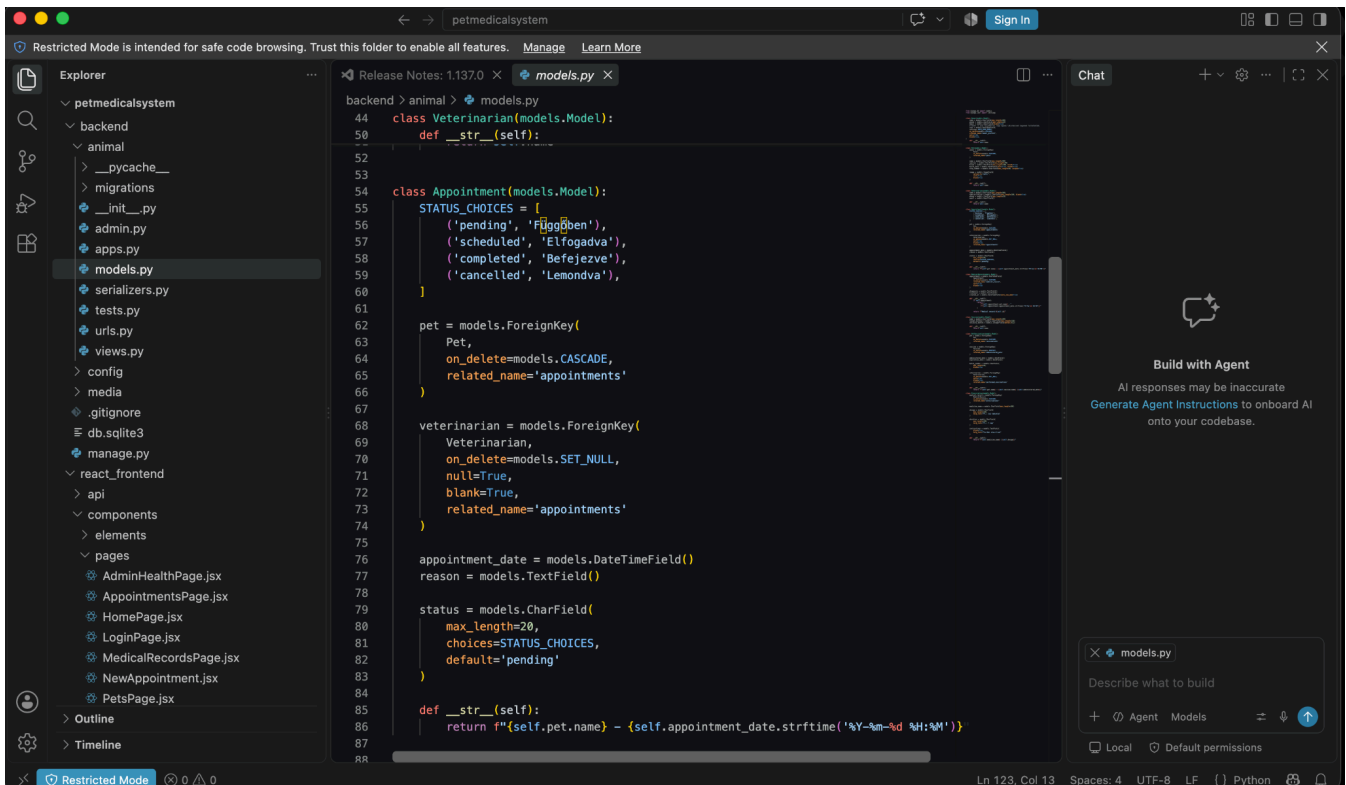
7.4 Appointment - időpont

Az Appointment modell a kisállathoz kapcsolódó időpontokat kezeli. A foglalás kisállathoz és opcionálisan állatorvoshoz kapcsolódik, továbbá tartalmazza a dátumot, a látogatás okát és az állapotot. Normál felhasználó új foglalása automatikusan pending állapotú, a státusz módosítása adminisztrátori jogosultsághoz kötött.

Státusz	Jelentés
pending	függőben, adminisztrátori elbírálásra vár
scheduled	elfogadva / ütemezve
completed	befejezve
cancelled	lemondva

7.4.1 Backend kódrészlet - Appointment modell

Az Appointment modellben a STATUS_CHOICES négy állapotot definiál. A pending az alapértelmezett érték, a scheduled az elfogadott időpontot, a completed a befejezett vizsgálatot, a cancelled pedig a lemondott foglalást jelöli.



11. ábra - Appointment modell és státuszkezelés a models.py fájlban

7.5 MedicalRecord - orvosi dokumentáció

A MedicalRecord egy konkrét Appointment rekordhoz kapcsolódik OneToOne kapcsolattal. A bejegyzés a diagnózist, a kezelést és a létrehozás dátumát tárolja. A kisállat adatai nem közvetlen pet_id mezőn keresztül, hanem az Appointment -> Pet kapcsolaton át érhetők el, így a vizsgálati esemény egyértelműen visszakereshető.

7.6 Vaccine és PetVaccination

Az oltási rendszer két külön fogalmat választ szét. A Vaccine a vakcinatípust reprezentálja, míg a PetVaccination egy konkrét állatnál megtörtént oltási eseményt. Ez csökkenti az ismétlődő adatokat és lehetővé teszi ugyanazon vakcinatípus több állathoz történő rögzítését.

- oltás típusa
- beadás dátuma
- lejárat dátum
- gyártási/tételszám
- oltást beadó állatorvos

7.7 Prescription - receptek

A Prescription modell nem közvetlenül a Pet modellhez, hanem a MedicalRecord rekordhoz kapcsolódik. Egy orvosi dokumentációhoz több gyógyszeres előírás is tartozhat. A recept a gyógyszer nevét, adagolását, a kezelés időtartamát és további utasításokat tárolja.

7.8 Adatmodell összefoglalása

Owner -> Pet: egy tulajdonoshoz több kisállat tartozhat.

Pet -> Appointment: egy kisállathoz több időpont tartozhat.

Veterinarian -> Appointment: egy állatorvos több időponthoz rendelhető.

Appointment -> MedicalRecord: egy időponthoz legfeljebb egy orvosi dokumentáció tartozik.

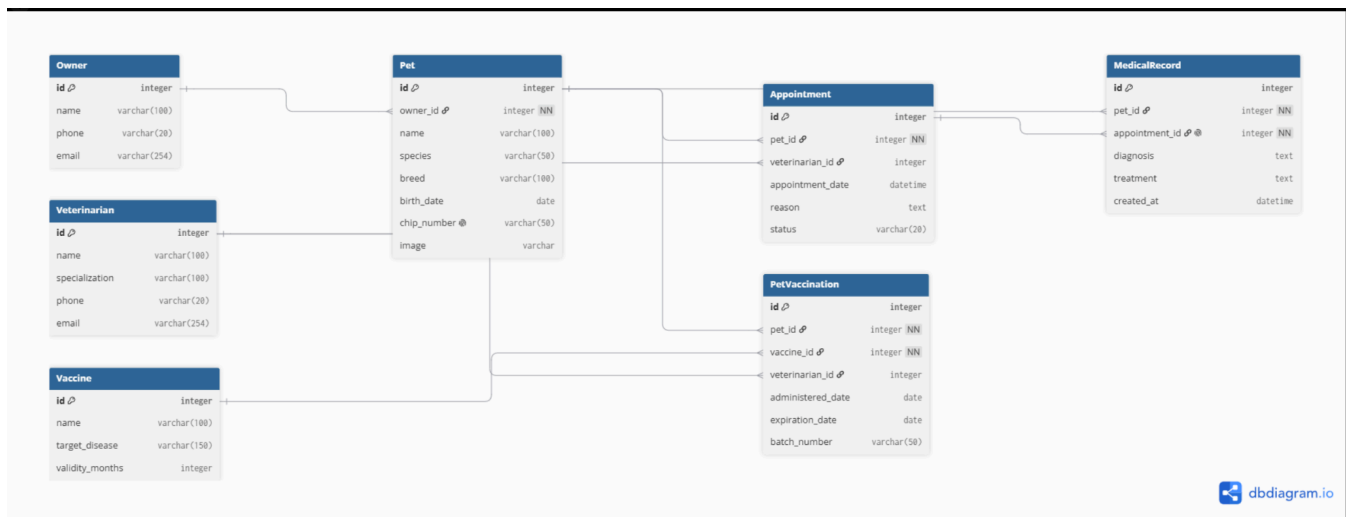
MedicalRecord -> Prescription: egy kezelési dokumentációhoz több recept tartozhat.

Pet / Vaccine / Veterinarian -> PetVaccination: az oltási esemény ezeket az adatokat kapcsolja össze.

A kapcsolati struktúra középpontjában továbbra is a kisállat áll, de az egészségügyi események a tényleges modellkapcsolatokon keresztül, egymásra épülve érhetők el.

7.9 Adatbázis-kapcsolati diagram

Az eredeti adatbázis-kapcsolati diagram a rendszer fő entitásait és azok kapcsolatait szemlélteti. Áttekinthetően mutatja be többek között az Owner, Pet, Veterinarian, Appointment, MedicalRecord, Vaccine és PetVaccination modellek közötti kapcsolatokat.



12. ábra - A backend eredeti adatbázis-kapcsolati diagramja

8. REST API

A backend egyik legfontosabb része a REST API. A Django REST Framework ModelViewSet és DefaultRouter megoldásai biztosítják az erőforrások kezelését.

Végpont	Erőforrás
/api/owners/	Tulajdonosok
/api/pets/	Kisállatok
/api/appointments/	Időpontok
/api/medical-records/	Orvosi dokumentáció
/api/veterinarians/	Állatorvosok
/api/vaccines/	Vakcinák
/api/pet-vaccinations/	Oltási események
/api/prescriptions/	Receptek

8.1 REST szemlélet

A REST API lehetővé teszi, hogy a frontend egységes módon kérjen le, hozzon létre vagy módosítson adatokat. A konkrét HTTP metódusok és végpontkonfigurációk a projekt forráskódjában ellenőrizhetők, az API-végpontok pedig az egyes klinikai erőforrások elkülönített kezelését biztosítják.

8.2 Példa API-folyamatokra

Kisállatok lekérdezése

A kliens a `/api/pets/` erőforrást használja a kisállatok adatainak elérésére. A válasz alapján a frontend kártyákon vagy listában jelenítheti meg az állatokat.

Időpontok kezelése

A `/api/appointments/` erőforrás az időpontfoglalások kezelésére szolgál. A felhasználói felületen az időpontok státusza is látható és adminisztratív szerepkörben módosítható.

Oltási adatok

A `/api/vaccines/` a vakcinatípusokat, a `/api/pet-vaccinations/` pedig a konkrét oltási eseményeket kezeli. Ez a két végpont együtt teszi lehetővé a klinikai oltási előzmények strukturált nyilvántartását.

Receptek

A `/api/prescriptions/` végpont az állathoz kapcsolódó gyógyszeres receptek frontend felé történő biztosítását támogatja.

9. Felhasználói hitelesítés és biztonság

A backend JWT-alapú hitelesítést használ a Django REST Framework Simple JWT segítségével. Bejelentkezéskor access és refresh token generálódik. A védett API végpontok IsAuthenticated jogosultság-ellenőrzést használnak.

Végpont	Feladat
POST /api/login/	Bejelentkezés és tokenek létrehozása
POST /api/token/refresh/	Access token frissítése

A kijelentkezéshez logout funkció készült, amely a refresh token érvénytelenítésével zárja le a munkamenetet.

9.1 Jogosulatlan hozzáférés

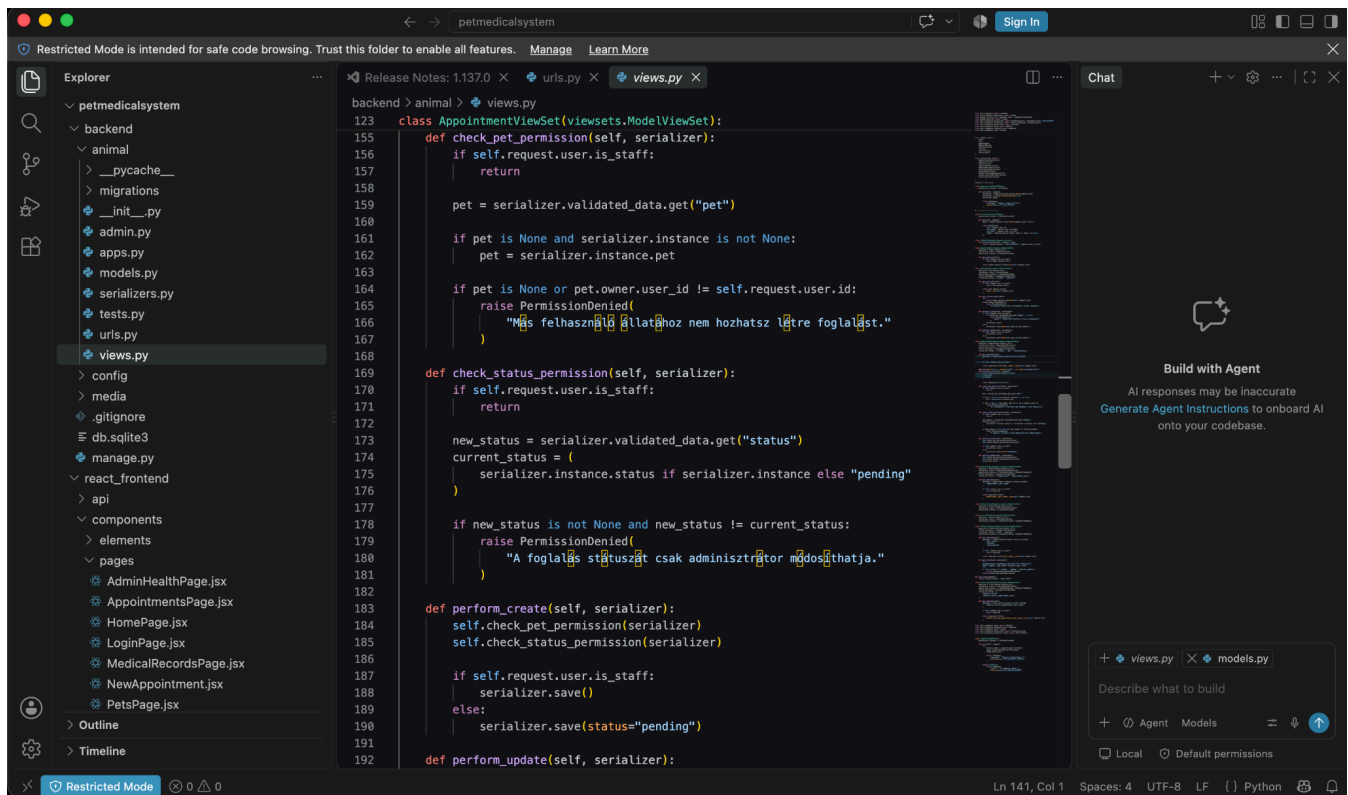
Hitelesítés nélkül a védett végpontok például a következő választ adhatják: {"detail": "Authentication credentials were not provided."}

9.2 Biztonsági szempontok

- védett API végpontok
- JWT tokenalapú munkamenet
- jogosultság-ellenőrzés
- Django keretrendszer beépített biztonsági mechanizmusai

9.3 Backend kódrészlet - foglalási jogosultságok

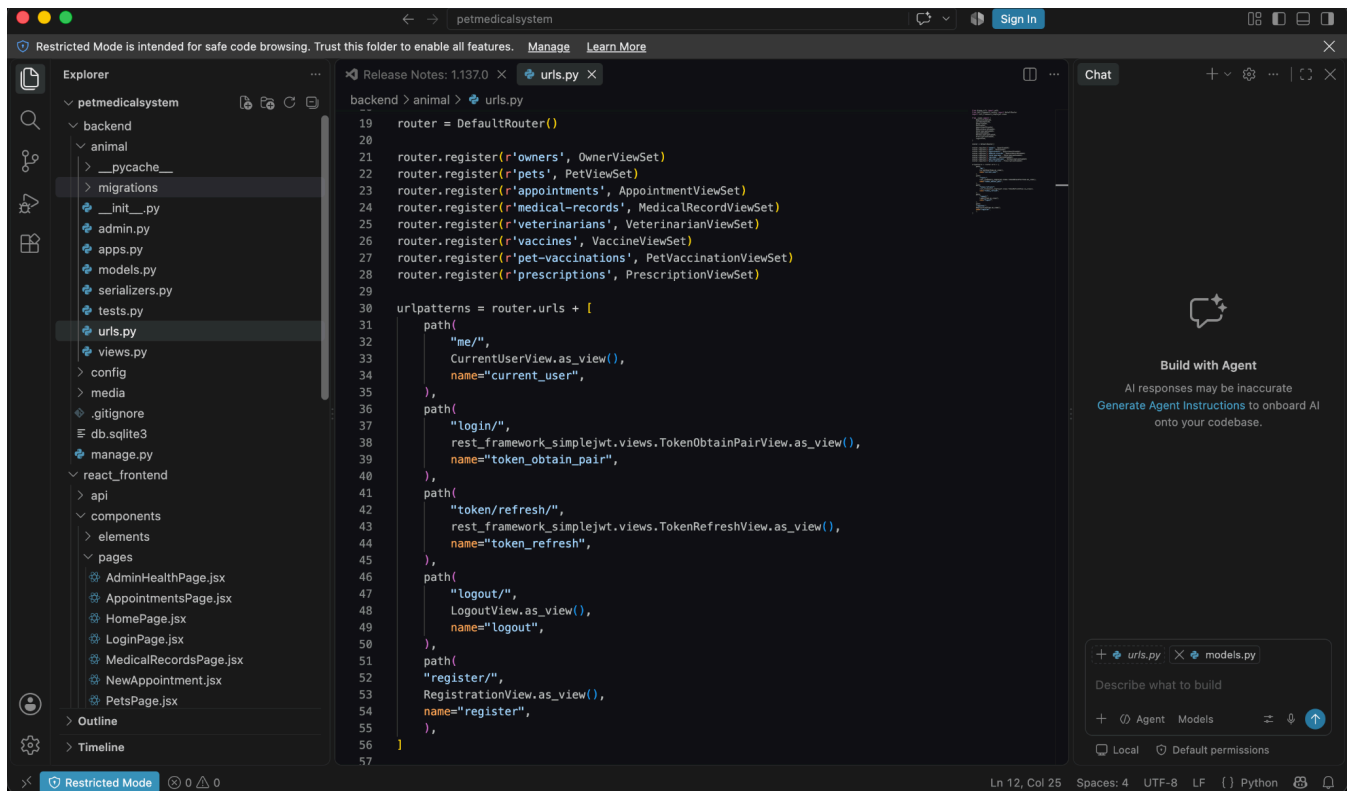
Az AppointmentViewSet üzleti logikája biztosítja, hogy normál felhasználó csak a saját kisállatához foglalhasson, és a foglalás státuszát ne módosíthassa. Új foglaláskor a backend automatikusan pending státuszt ment.



13. ábra - Időpontfoglaláshoz kapcsolódó jogosultságkezelés a backendben

9.4 Backend kódrészlet - REST API és JWT végpontok

A DefaultRouter regisztrálja a fő klinikai erőforrások ViewSetjeit. A külön útvonalak kezelik az aktuális felhasználó lekérdezését, a JWT bejelentkezést, tokenfrissítést, kijelentkezést és regisztrációt.



14. ábra - REST API router és hitelesítési végpontok

10. Adminisztrációs felület

A rendszer a Django beépített adminisztrációs felületét is használja. Az adminisztrátor innen kezelheti a tulajdonosokat, kisállatokat, állatorvosokat, időpontokat, orvosi dokumentációkat és oltásokat.

Az adminisztráció különösen fontos az időpontfoglalások kezelésénél, mert az adminisztrátor ellenőrizheti és módosíthatja az időpontok állapotát.

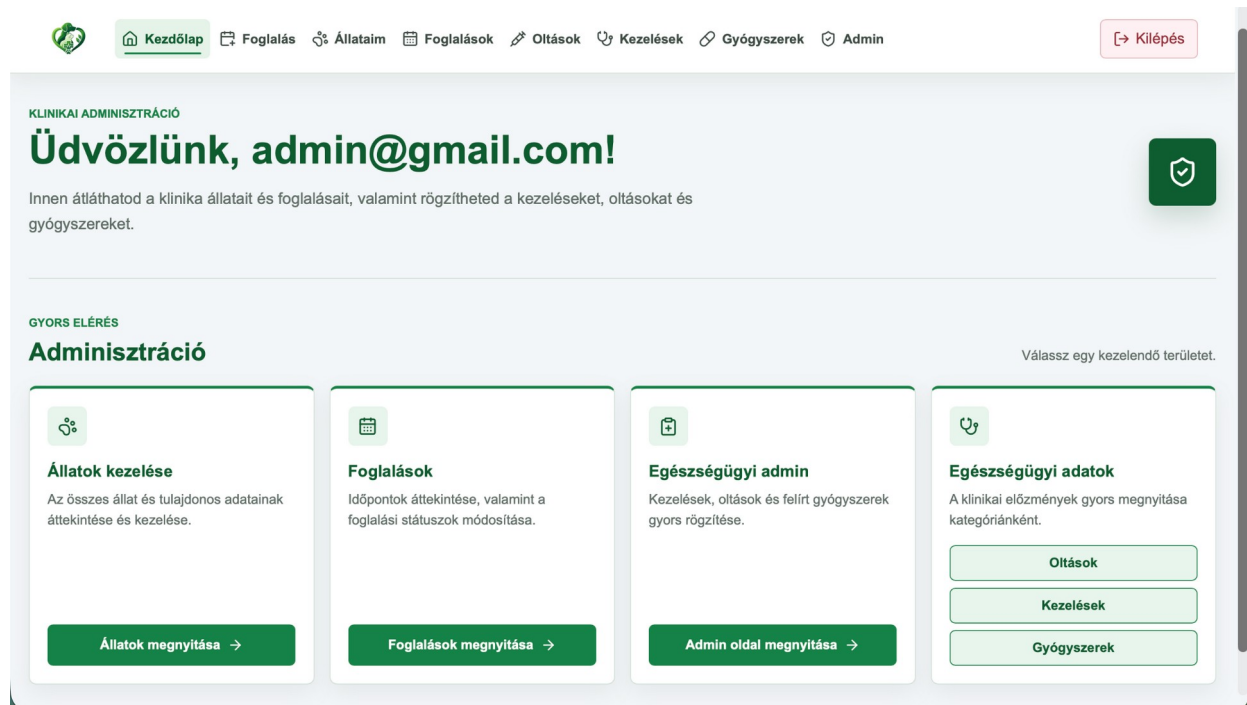
10.1 Adminisztráció előnyei

- Gyors adatellenőrzés fejlesztés és tesztelés közben.
- A modellek rekordjainak központi kezelése.
- Időpontok és státuszok felügyelete.
- A frontendtől független adminisztratív hozzáférés.

A Django admin nem helyettesíti a végfelhasználói felületet, hanem kiegészítő, magasabb jogosultságú kezelőeszköz.

11. Frontend fejlesztés és felhasználói felület

A kliensoldali alkalmazás React 19 és Vite alapokra épül. A frontend feladata, hogy a Django REST API által biztosított adatokat könnyen kezelhető, magyar nyelvű webes felületen jelenítse meg. A kód külön oldal-, elem- és szolgáltatáskomponensekre tagolódik, így az egyes funkciók jól elkülönülnek.



1. ábra - Adminisztrátori kezdőoldal

A felső navigációból közvetlenül elérhető a Kezdőlap, Foglalás, Állataim, Foglalások, Oltások, Kezelések, Gyógyszerek és - megfelelő jogosultság esetén - az Admin oldal. Az adminisztrátori kezdőlapon gyorshivatkozások segítik a legfontosabb klinikai műveleteket.

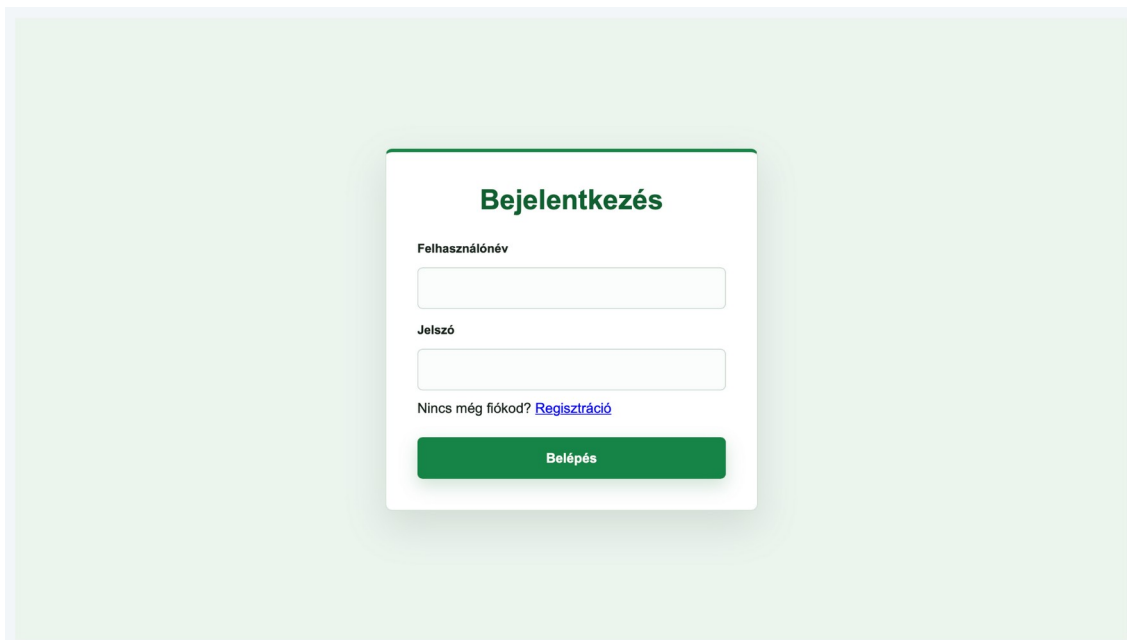
11.1 Frontend technológiák és projektstruktúra

A frontend fő függőségei a react, react-dom, react-router-dom és lucide-react csomagok. A Vite fejlesztői szerver, gyors újratöltést és build folyamatot biztosít. A CSS fájlok funkcionális oldalak szerint különülnek el, például külön stíluslap tartozik a bejelentkezéshez, a foglalásokhoz, a kisállatokhoz és az adminisztrációs felülethez.

A projekt legfontosabb könyvtárai: src - alkalmazás belépési pontja és fő komponense; components/pages - oldalkomponensek; components/elements - újrahazsnosítható űrlapok; components/services - hitelesítési segédfüggvények; api - a backenddel kommunikáló függvények; styles - CSS stíluslapok; public - logó és webes ikonok.

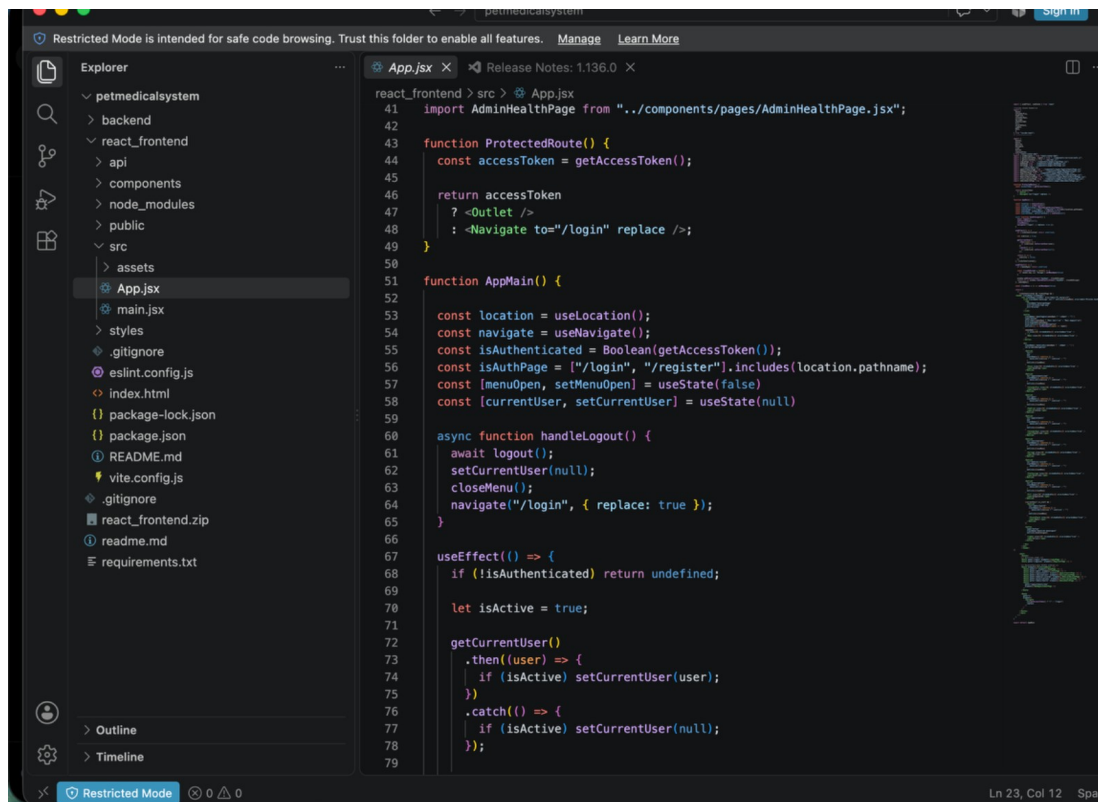
11.2 Routing és védett útvonalak

Az útvonalkezelést a react-router-dom végzi. A /login és /register oldalak nyilvánosan elérhetők, a többi fontos oldal ProtectedRoute komponensen keresztül érhető el. Ha nincs érvényes access token, a rendszer a bejelentkezési oldalra irányítja a felhasználót. A nem létező útvonalak szintén a megfelelő kezdőoldalra vagy bejelentkezéshez vezetnek vissza.



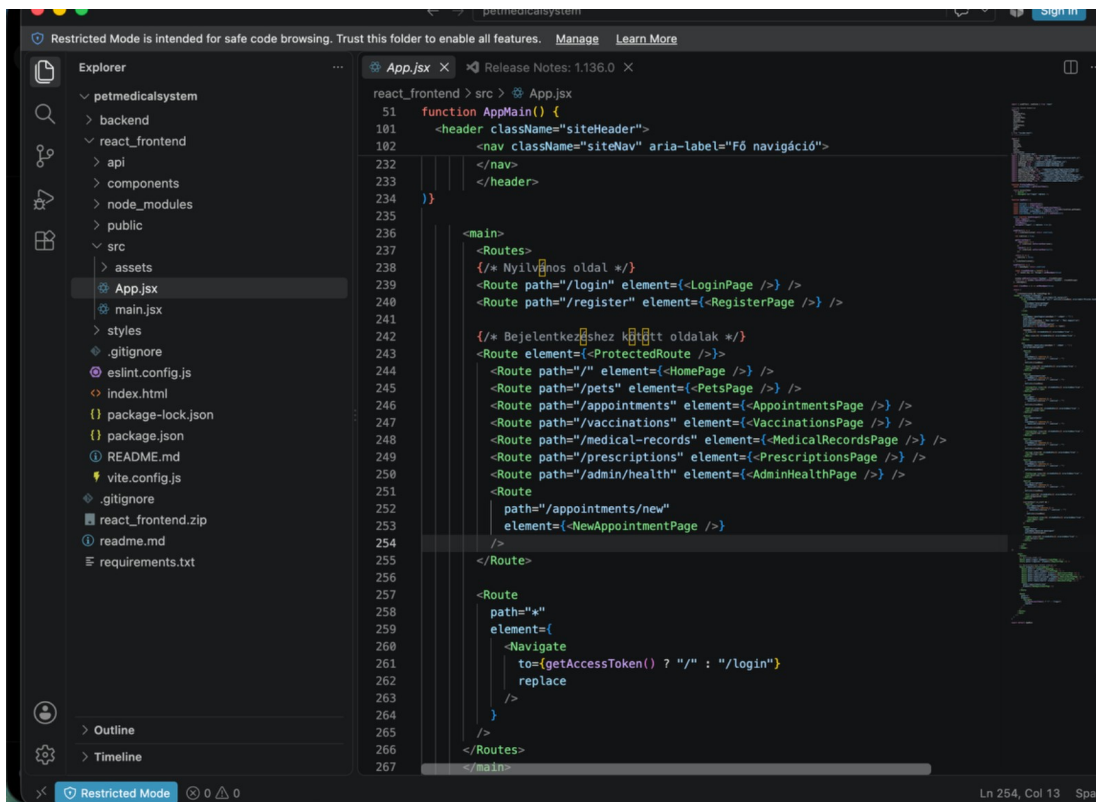
2. ábra - Bejelentkezési felület

A bejelentkezési űrlap felhasználónevet és jelszót kér. Sikeres azonosítás után a felhasználó a kezdőoldalra kerül, hibánál pedig a felület szöveges visszajelzést jelenít meg. A regisztráció közvetlenül a bejelentkezési oldalról érhető el.



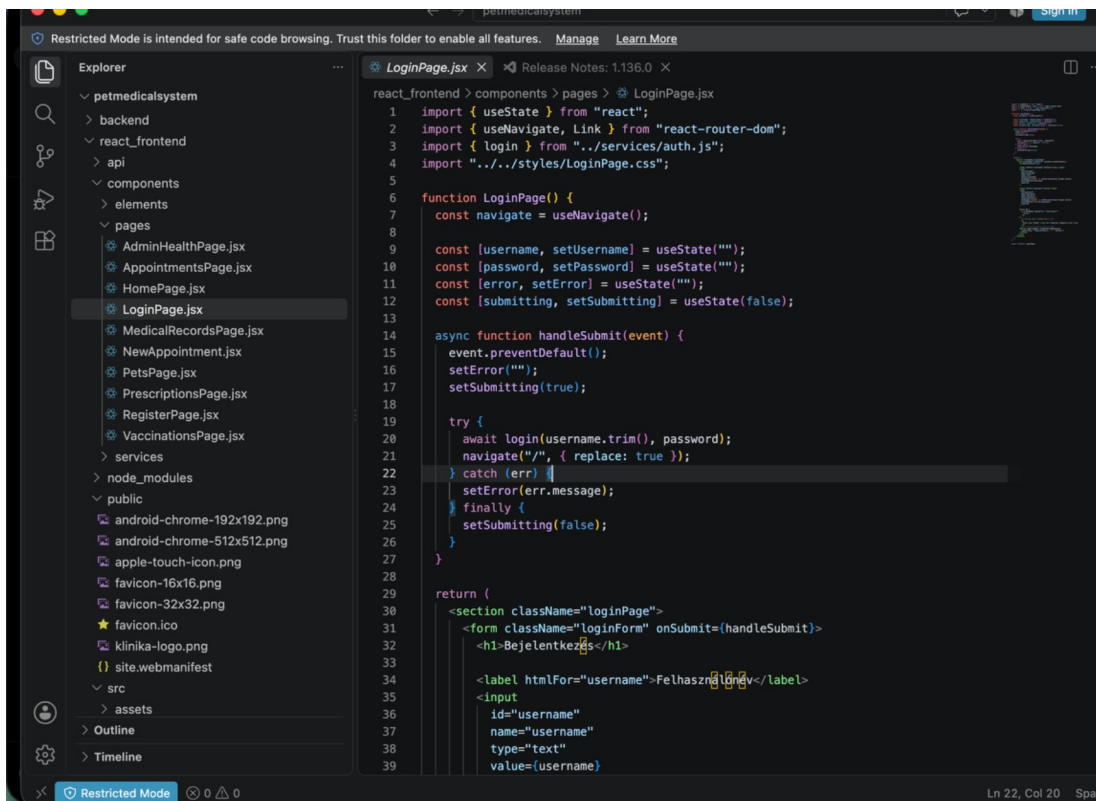
5. ábra - Védett útvonal és autentikáció az App.jsx fájlban

A ProtectedRoute komponens az access token megléte alapján engedélyezi a védett oldalak elérését. Ha a felhasználó nincs bejelentkezve, a rendszer a bejelentkezési oldalra irányítja.



6. ábra - React Router útvonalak az App.jsx fájlban

Az App.jsx útvonal-konfigurációja elkülöníti a nyilvános és a hitelesítéshez kötött oldalakat. A védett útvonalak egységes alkalmazáselrendezésen keresztül érhetők el.



7. ábra - Bejelentkezési

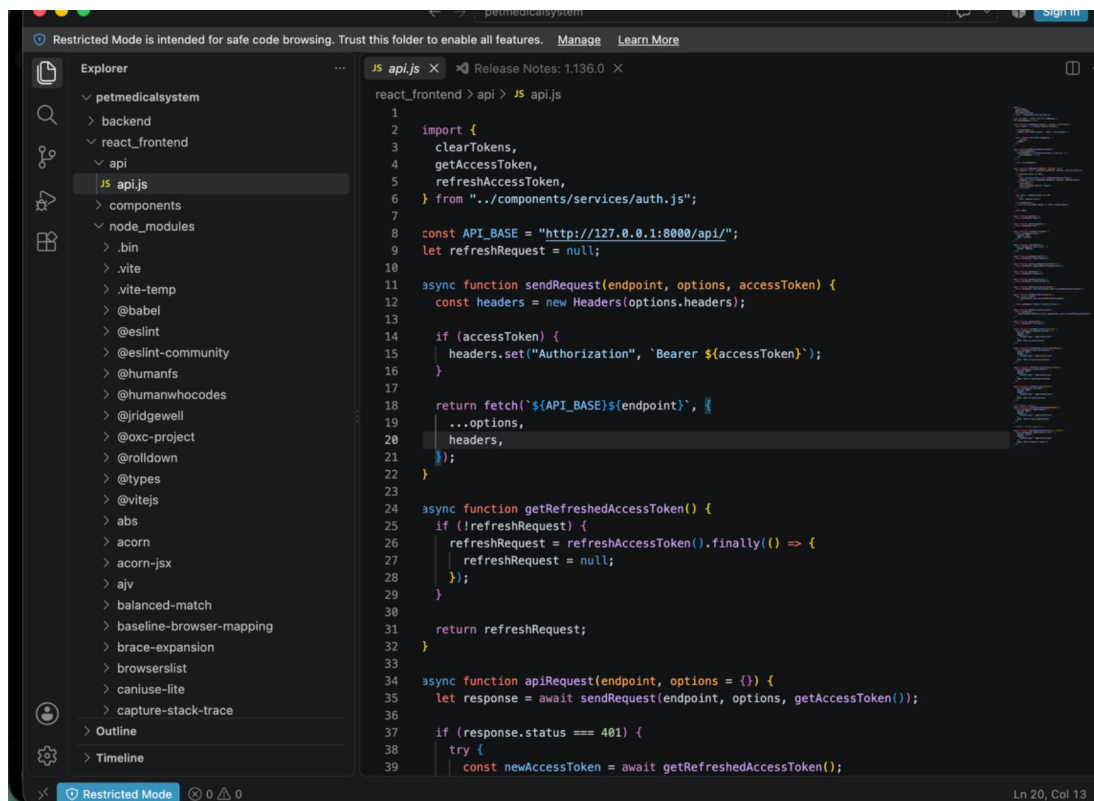
a LoginPage.jsx fájlban

A bejelentkezési komponens állapotváltozókkal kezeli a felhasználónevet, a jelszót, a beküldési állapotot és az esetleges hibaüzenetet. Sikeres hitelesítés után a felhasználó a kezdőoldalra kerül.

11.3 JWT-kezelés és API-kommunikáció

A frontend központi API-rétege a `http://127.0.0.1:8000/api/` alaplímet használja. A kérések Authorization fejlécében Bearer access token továbbítható. Ha a szerver 401-es választ ad, a kliens megkísérli a token frissítését a refresh token segítségével, majd megismétli az eredeti kérést. Sikertelen frissítés esetén a tokenek törlődnek, és a felhasználó visszakerül a bejelentkezési oldalra.

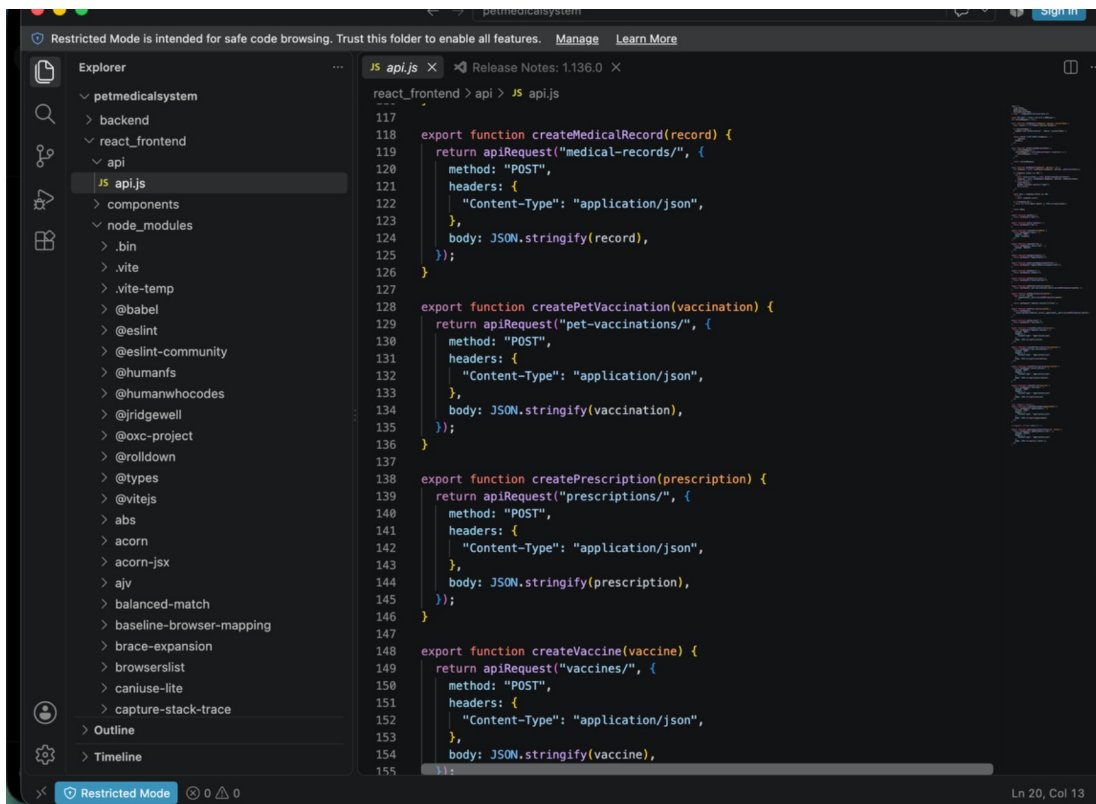
A tokenek kliensoldali tárolását és a bejelentkezési/kijelentkezési műveleteket külön auth szolgáltatás kezeli. Ez csökkenti az ismétlődő kód mennyiségét, és egységesíti az autentikáció működését az egész frontendben.



```
1 import {
2   clearTokens,
3   getAccessToken,
4   refreshAccessToken,
5 } from "../components/services/auth.js";
6
7 const API_BASE = "http://127.0.0.1:8000/api/";
8 let refreshRequest = null;
9
10 async function sendRequest(endpoint, options, accessToken) {
11   const headers = new Headers(options.headers);
12
13   if (accessToken) {
14     headers.set("Authorization", `Bearer ${accessToken}`);
15   }
16
17   return fetch(`${API_BASE}${endpoint}`, {
18     ...options,
19     headers,
20   });
21 }
22
23 async function getRefreshedAccessToken() {
24   if (!refreshRequest) {
25     refreshRequest = refreshAccessToken().finally(() => {
26       refreshRequest = null;
27     });
28   }
29
30   return refreshRequest;
31 }
32
33 async function apiRequest(endpoint, options = {}) {
34   let response = await sendRequest(endpoint, options, getAccessToken());
35
36   if (response.status === 401) {
37     try {
38       const newAccessToken = await getRefreshedAccessToken();
39     }
```

8. ábra - JWT token és API-kérések kezelése az `api.js` fájlban

A központi API-segédfüggvény automatikusan hozzáadja a Bearer token a kérésekhez. Lejárt access token esetén a kliens megkísérli a token frissítését, majd újaküldi az eredeti kérést.

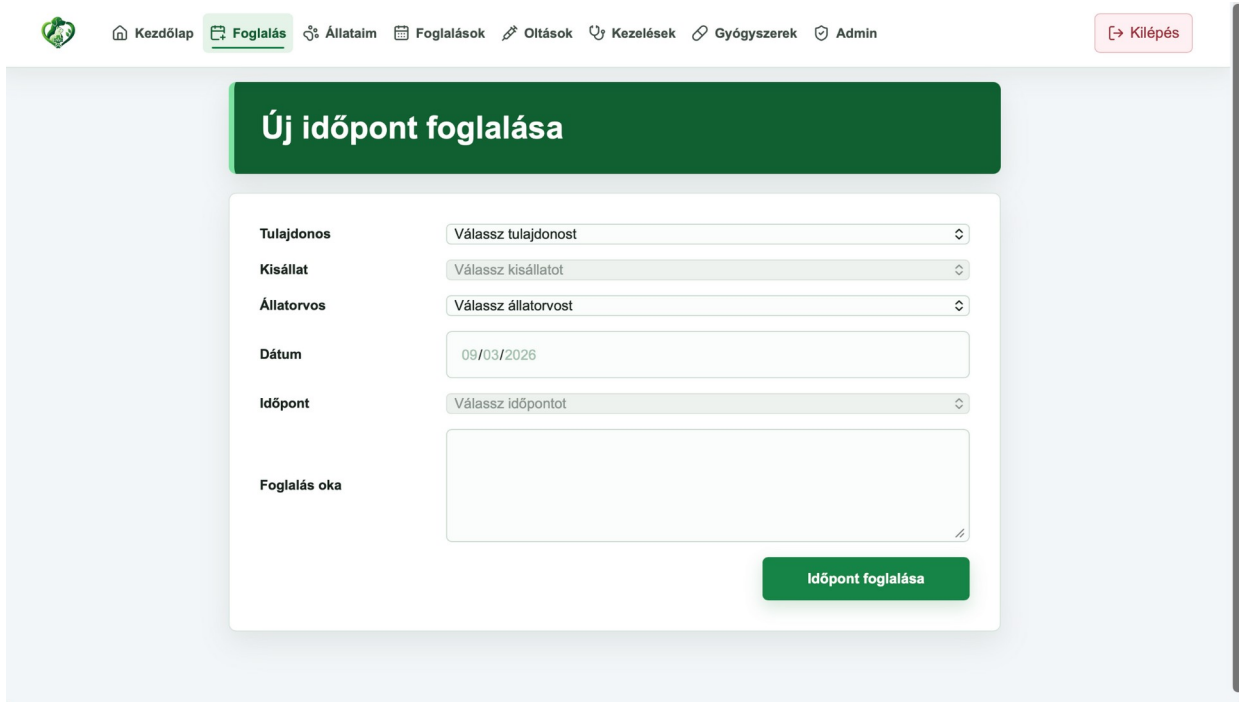


```
react_frontend > api > JS api.js
117
118 export function createMedicalRecord(record) {
119   return apiRequest("medical-records/", {
120     method: "POST",
121     headers: {
122       "Content-Type": "application/json",
123     },
124     body: JSON.stringify(record),
125   });
126 }
127
128 export function createPetVaccination(vaccination) {
129   return apiRequest("pet-vaccinations/", {
130     method: "POST",
131     headers: {
132       "Content-Type": "application/json",
133     },
134     body: JSON.stringify(vaccination),
135   });
136 }
137
138 export function createPrescription(prescription) {
139   return apiRequest("prescriptions/", {
140     method: "POST",
141     headers: {
142       "Content-Type": "application/json",
143     },
144     body: JSON.stringify(prescription),
145   });
146 }
147
148 export function createVaccine(vaccine) {
149   return apiRequest("vaccines/", {
150     method: "POST",
151     headers: {
152       "Content-Type": "application/json",
153     },
154     body: JSON.stringify(vaccine),
155   });
156 }
```

9. ábra - Adatrögzítő POST műveletek az api.js fájlban

Az api.js külön függvényeket biztosít az egészségügyi adatok létrehozására. A közös apiRequest használata egységesíti a szerver felé küldött POST kérések kezelését.

11.4 Időpontfoglalás



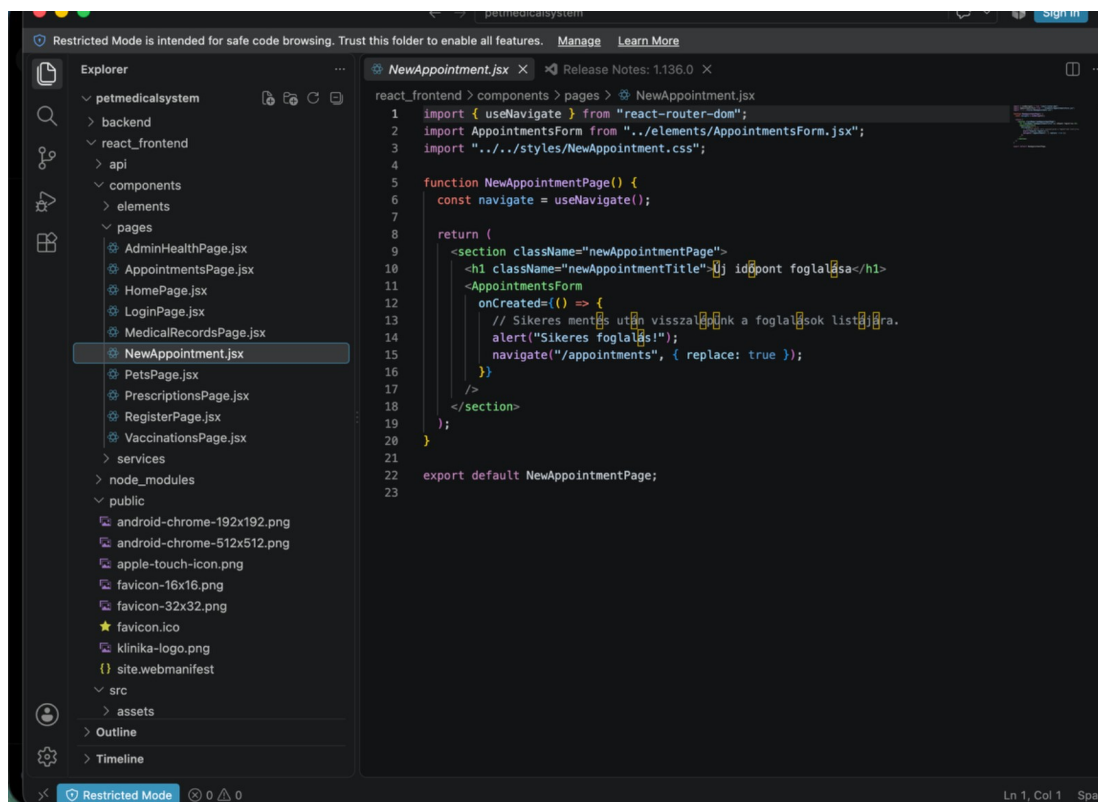
The screenshot shows a web application interface for booking an appointment. At the top, there is a navigation bar with links: Kezdőlap, Foglалás (highlighted), Állataim, Foglалások, Oltások, Kezelések, Gyógyszerek, and Admin. A 'Kilépés' (Logout) button is on the right. The main heading is 'Új időpont foglalása'. Below it is a form with the following fields:

- Tulajdonos: Válassz tulajdonost (dropdown)
- Kisállat: Válassz kisállatot (dropdown)
- Állatorvos: Válassz állatorvost (dropdown)
- Dátum: 09/03/2026 (text input)
- Időpont: Válassz időpontot (dropdown)
- Foglalás oka: (text area)

A green 'Időpont foglalása' button is located at the bottom right of the form.

3. ábra - Új időpont foglalása

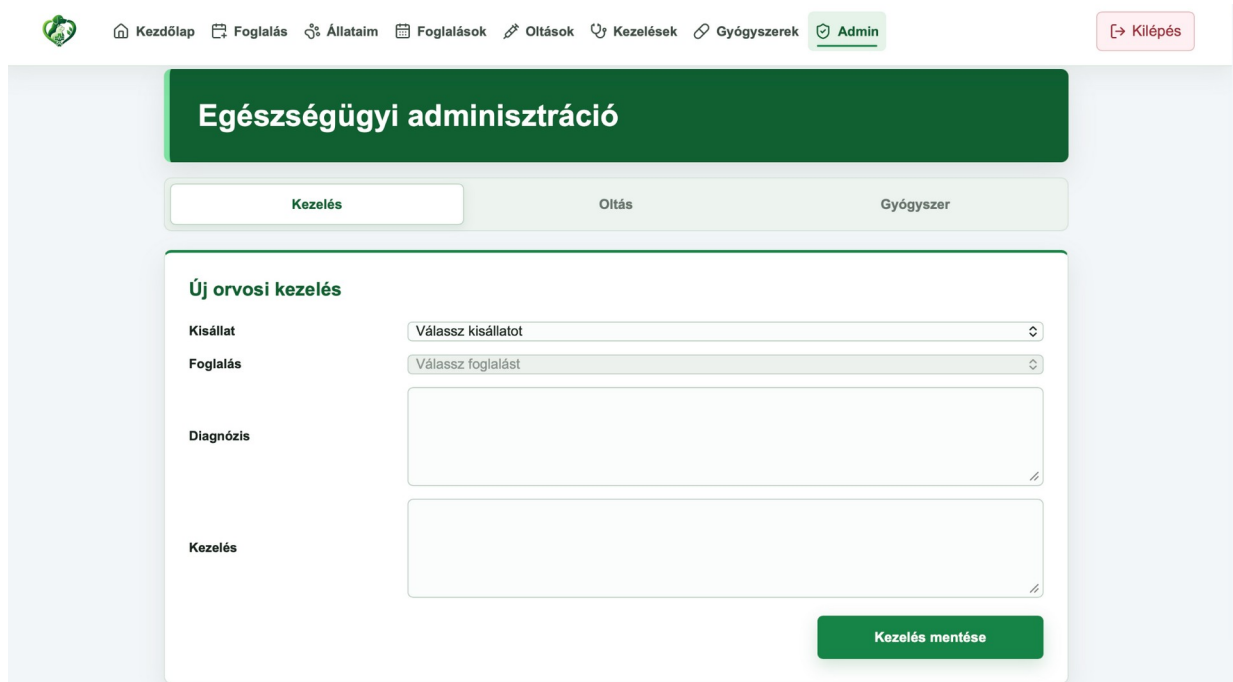
Az új időpont űrlap tulajdonos, kisállat, állatorvos, dátum, időpont és foglalási ok adatait kezeli. A kapcsolódó mezők egymásra épülnek: a kisállat kiválasztása például csak a tulajdonos megadását követően válik értelmezhetővé. A mentés előtt a szükséges adatokat a kliens ellenőrzi, majd API-kérésben továbbítja a backend felé.



10. ábra - Az új időpontfoglaló oldal komponense

A NewAppointment komponens az AppointmentForm űrlapot használja. Sikeres létrehozás után visszajelzést ad a felhasználónak, majd a foglalások oldalára navigál.

11.5 Adminisztrációs frontend



4. ábra - Egészségügyi adminisztráció

Az adminisztrációs oldal három fő területet fog össze: kezelés, oltás és gyógyszer. A felület külön űrlapkomponenseket használ az orvosi kezelés, az oltási esemény és a gyógyszerfelírás rögzítésére. Az Admin menüpont csak akkor jelenik meg, ha az aktuális felhasználó is_staff jogosultsággal rendelkezik.

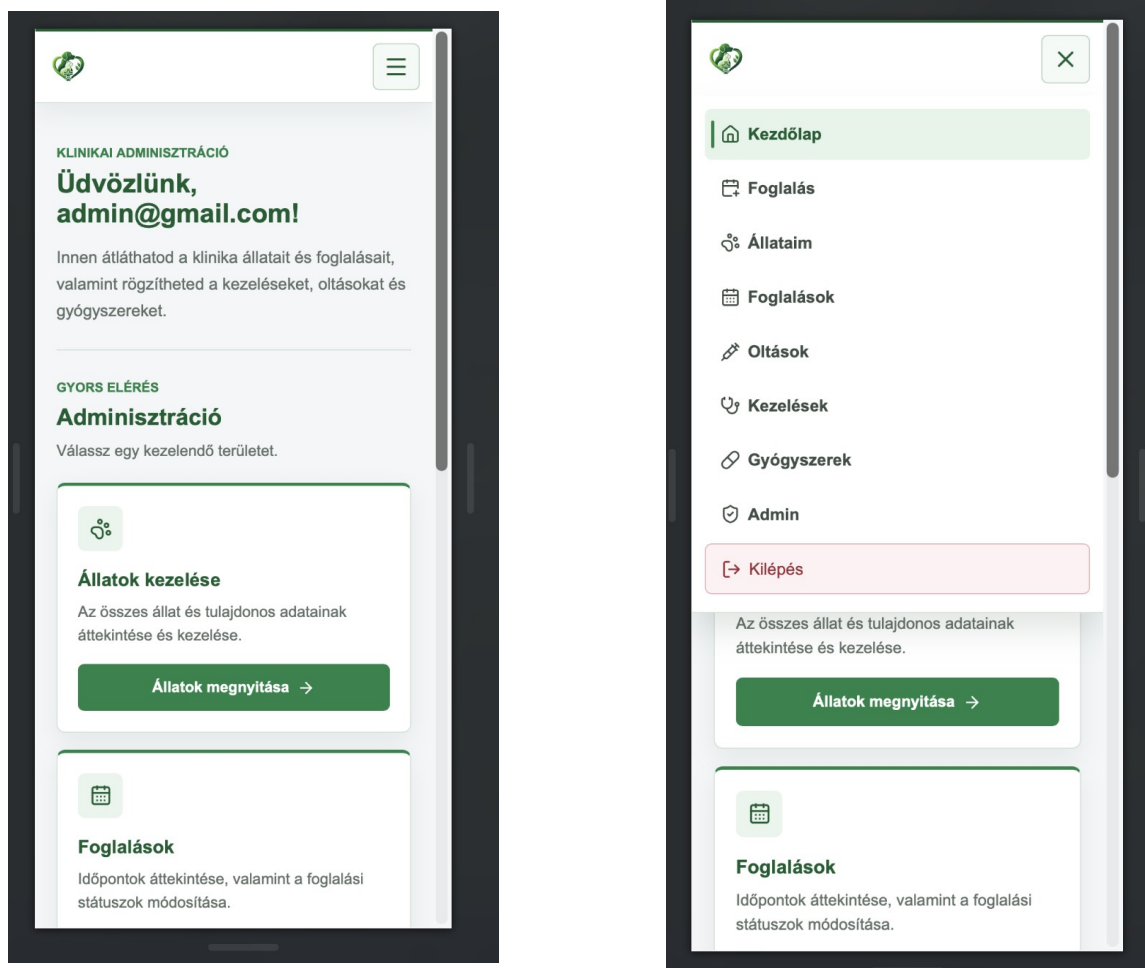
11.6 Állapotkezelés és felhasználói visszajelzés

Az oldalak a React useState és useEffect hookjait használják az űrlapadatok, a lekért listák, a betöltési állapot és a hibák kezelésére. Az aszinkron API-hívások eredményét a komponensek állapotváltozóba mentik, majd ennek megfelelően frissítik a felületet. Betöltés közben és hiba esetén külön visszajelzés jelenhet meg.

A navigáció reszponzív kialakítású: kisebb képernyőn nyitható/zárható menü működik, amely az Escape billentyűvel is bezárható. A lucide-react ikonok vizuálisan segítik az egyes funkciók azonosítását.

11.7 Reszponzív megjelenítés

A felhasználói felület reszponzív kialakítású, így az alkalmazás elrendezése automatikusan alkalmazkodik a rendelkezésre álló képernyőmérethez. Kisebb kijelzőn a tartalmi elemek egy oszlopba rendeződnek, a navigáció pedig kompakt hamburger menüre vált. A mobilnézet kialakítása biztosítja, hogy az alkalmazás fő funkciói kisebb kijelzőn is könnyen elérhetők és használhatók legyenek.



11. ábra - Az alkalmazás reszponzív mobilnézete és a mobil navigáció

12. A program használata

12.1 Bejelentkezés és kezdőoldal

A rendszer használata a bejelentkezéssel kezdődik. Sikeres belépés után a felhasználó saját jogosultságának megfelelő kezdőoldalt kap. Normál felhasználó a saját kisállataihoz és egészségügyi adataihoz fér hozzá, az adminisztrátor pedig a klinika teljesebb kezelőfelületét használhatja.

12.2 Kisállatok és foglалások kezelése

A regisztrált állattulajdonos az Állataim oldalon a saját kisállatait tekintheti meg és új állatot is rögzíthet. A Foglалás menüpontban kizárólag saját állatához kezdeményezhet időpontot. Az új foglалás pending (függőben) státusszal jön létre; annak állapotát az adminisztrátor bírálhatja el és módosíthatja.

12.3 Oltások, kezelések és gyógyszerek

Az Oltások, Kezelések és Gyógyszerek menüpontok a kisállatok egészségügyi előzményeit külön kategóriákban jelenítik meg. Adminisztrátori jogosultsággal az Egészségügyi adminisztráció oldalon új kezelési adat, oltási esemény vagy gyógyszerfelírás rögzíthető.

12.4 Felhasználói folyamat és jogosultságok

A rendszer két eltérő használati módot támogat. A normál felhasználó saját állataihoz és az azokhoz tartozó adatokhoz fér hozzá, míg az adminisztrátor a teljesebb klinikai adatállományt kezelheti. A backend minden fontos műveletnél ellenőrzi a bejelentkezést és a tulajdonosi kapcsolatot.

1. Regisztráció: a felhasználói fiók létrehozásakor automatikusan Owner profil is készül.
2. Kisállat felvétele: a normál felhasználó a saját Owner profiljához rögzíthet kisállatot.
3. Időpontfoglalás: a felhasználó csak a saját állatához hozhat létre foglalást.
4. Függőben lévő állapot: a foglalás normál felhasználónál automatikusan pending státuszt kap.
5. Adminisztrátori elbírálás: a státuszt csak adminisztrátor módosíthatja, például scheduled, completed vagy cancelled értékre.
6. Adatvédelem: a normál felhasználó lekérdezései a saját Owner profiljához tartozó rekordokra szűródnék.

12.4 Kijelentkezés

A Kilépés gomb lezárja a felhasználói munkamenetet. A kliens törli a tárolt access és refresh tokeneket, majd visszairányítja a felhasználót a bejelentkezési oldalra.

Modul	Backend kapcsolat	Cél
Oltások	Vaccine, PetVaccination	Vakcinatípus és beadási esemény nyilvántartása
Kezelések	MedicalRecord	Diagnózisok és kezelések rögzítése
Gyógyszerek	Prescription	Gyógyszeres receptek kezelése

13. Telepítési és futtatási útmutató

A rendszer helyi futtatása Windows és macOS környezetben is lehetséges. A vizsgán elsődlegesen a Windows alapú indítási folyamatot mutatjuk be, majd külön szerepel a macOS változat. A backend és a frontend két külön terminálablakban fut.

13.1 Szükséges szoftverek

Git, Python 3 és pip, Node.js és npm, valamint egy modern böngésző szükséges. A forráskód szerkesztéséhez Visual Studio Code használható. A csomagverziókat a requirements.txt és a react_frontend/package.json fájlok határozzák meg.

13.2 Projekt letöltése

A projekt Git repositoryból klónozzható, majd a petmedicalsistem könyvtárba kell belépni. A gyökérmappában külön backend és react_frontend könyvtár található.

```
git clone <repository-címe>
cd petmedicalsistem
```

13.3 Backend indítása Windows alatt

A következő parancsok PowerShellben vagy Parancssorban futtathatók. A virtuális környezet elkülöníti a projekthez szükséges Python csomagokat a gép többi Python telepítésétől.

```
py -m venv venv
venv\Scripts\activate
py -m pip install -r requirements.txt
cd backend
py manage.py migrate
py manage.py runserver
```

A backend sikeres indítás után alapértelmezetten a `http://127.0.0.1:8000` címen fut. A REST végpontok az `/api/` útvonal alatt érhetők el. Ha a gyökérút nincs külön definiálva, a fő címen megjelenő 404 oldal nem jelenti azt, hogy a szerver nem működik.

13.4 Frontend indítása Windows alatt

Új PowerShell vagy Parancssor ablakban a projekt frontend könyvtárába kell lépni, telepíteni a JavaScript függőségeket, majd elindítani a Vite fejlesztői szerveret.

```
cd petmedicalsystem\react_frontend
npm install
npm run dev
```

A terminál megjeleníti a frontend helyi címét. Vite esetén ez jellemzően `http://localhost:5173`, de mindig a terminálban kiírt aktuális címet kell megnyitni a böngészőben.

13.5 Futtatás macOS alatt

macOS alatt a folyamat megegyezik, de a Python és a virtuális környezet aktiválásának parancsai eltérnek. A backendhez a projekt gyökérmappájából az alábbi parancsok használhatók:

```
python3 -m venv venv
source venv/bin/activate
python3 -m pip install -r requirements.txt
cd backend
python3 manage.py migrate
python3 manage.py runserver
```

A frontend indítása macOS alatt ugyanúgy `npm install`, majd `npm run dev` paranccsal történik a `react_frontend` könyvtárban.

13.6 A rendszer leállítása

A fejlesztői szerverek a terminálban `Ctrl+C` billentyűkombinációval állíthatók le. Újraindításkor a függőségek telepítését nem kell minden alkalommal megismételni; elegendő a virtuális környezet aktiválása, majd a backend és frontend indítása.

14. Tesztdokumentáció

A vizsgaremek tesztelése funkcionális, jogosultsági, adatkezelési és felhasználói felületi ellenőrzéseket foglal magában. A tesztesetek a rendszer ténylegesen megvalósított funkcióit vizsgálják.

ID	Teszt	Elvárt eredmény
T01	Érvényes bejelentkezés	Access és refresh token létrejön
T02	Védett API token nélkül	Hozzáférés megtagadva
T03	Tulajdonos létrehozása	Új Owner rekord létrejön
T04	Kisállat tulajdonoshoz rendelése	Kapcsolat mentődik
T05	Kisállat kép feltöltése	Kép mentődik és megjeleníthető
T06	Időpont létrehozása	Appointment rekord létrejön
T07	Időpont státusz módosítása	Új státusz megmarad
T08	Oltási esemény rögzítése	PetVaccination mentődik
T09	Kezelési előzmény rögzítése	MedicalRecord létrejön
T10	Recept rögzítése	Prescription létrejön

14.1 Felhasználói felület tesztelése

Terület	Ellenőrzés
Navigáció	Minden menüpont a megfelelő oldalra visz.
Foglalás	A szükséges mezők kiválaszthatók és a foglalás elküldhető.
Állataim	Új állat felvehető, a meglévők megjelennek.
Foglalások	A táblázat adatai és státuszai olvashatók.
Kijelentkezés	A munkamenet lezárul.
Hibakezelés	Hibás vagy hiányzó adat esetén érthető visszajelzés jelenik meg.

14.2 API-tesztelés

Az API végpontok tesztelhetők böngészhető DRF felületen vagy API-klienssel. Különösen fontos a jogosultsági ellenőrzés: a védett végpontok token nélkül ne adjanak hozzáférést a klinikai adatokhoz.

14.3 Tesztadatok

A teszteléshez demonstrációs felhasználók és klinikai adatok használhatók. Külön tesztfelhasználók alkalmazásával ellenőrizhető a normál felhasználói és az adminisztrátori jogosultság közötti különbség.

15. Továbbfejlesztési lehetőségek

A jelenlegi rendszer jó alapot biztosít egy összetettebb klinikai alkalmazáshoz. Az alábbi funkciók nem részei a jelenlegi verciónak, de logikus továbbfejlesztési irányok.

- Szerepkör-alapú jogosultságok finomítása külön adminisztrátor, recepció és állatorvos szereppel.
- Automatikus e-mailes vagy alkalmazáson belüli értesítés a foglaltság státuszának változásáról, például elfogadásról vagy lemondásról.
- Időpont-émlekeztetők és lejáró oltások automatikus figyelése.
- Felhasználói profil kezelése: név, e-mail-cím, telefonszám és más saját fiókadatak módosítása.
- Részletes keresés és szűrés tulajdonos, állat, dátum és státusz szerint.
- PDF-formátumú kezelési lap vagy recept export.
- PostgreSQL adatbázis használata élesebb, többfelhasználós környezetben.
- Automatizált backend és frontend tesztek, auditnapló és további mobilnézeti finomítások.

15.1 Lehetséges frontend fejlesztések

A frontend továbbfejlesztése során egységes értesítési központ, részletesebb űrlapvalidáció, saját profil szerkesztőfelület, valamint automatizált komponens- és felhasználói felületi tesztek vezethetők be.

16. Összefoglalás

Az Állatorvosi Klinika Kezelőrendszer egy többmodulos webalkalmazás, amely a rendelői adminisztráció és az állattulajdonosi önkiszolgálás legfontosabb folyamatait egy közös rendszerben kezeli. A Django és Django REST Framework alapú backend strukturált adatmodelleket, REST API-t és jogosultsági ellenőrzést biztosít.

A regisztrált állattulajdonos saját profiljához kapcsolódó kisállatokat kezelhet és időpontot kezdeményezhet. A foglалás függőben lévő státusszal jön létre, amelyet az adminisztrátor bírálhat el. A rendszer emellett kezeli az állatorvosokat, orvosi dokumentációkat, vakcinákat, oltási eseményeket és a MedicalRecordhoz kapcsolódó recepteket.

A JWT-alapú hitelesítés védi az API-végpontokat, a React-alapú kliens pedig reszponzív, többfelhasználós felületet biztosít Windows és macOS fejlesztői környezetben egyaránt.

Irodalomjegyzék

<https://www.w3schools.com/>

<https://react.dev/learn>

<https://www.djangoproject.com/start/>